202044502
Programming with
JAVA

Unit -1
Topic – O.O, Class, Object

Academic Year: 2024-25 (II)

Prepared By:
Prof. Pratik B. Soni
(Asst. Prof. – I.T. Dept, MBIT)

Object - Oriented

- ***Encapsulation*** is the mechanism that binds together code and the data it manipulates, and keeps both safe from outside interference and misuse.

- ***Inheritance*** is the process by which one object acquires the properties of another object. This is important because it supports the concept of hierarchical classification.

- ***Polymorphism*** (from Greek, meaning “many forms”) is a feature that allows one interface to be used for a general class of actions.

Class

```
class classname {
        type instance-variable1;
        type instance-variable2;
        // ...
        type instance-variableN;

        type methodname1(parameter-list)
        {
                // body of method
        }
        type methodname2(parameter-list)
        {
                // body of method
        }
}
```

- The data, or variables, defined within a **class** are called *instance variables.*
- The code is contained within *methods.* Collectively, the methods and variables defined within a class are called *members* of the class.
- Class Members are accessed by **class Objects**

Object

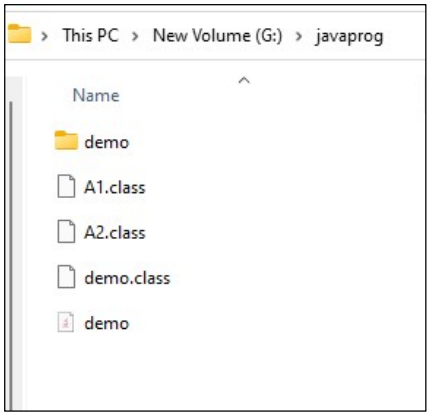
Instance of class
Class Instantiation
Allocation of Memory to class

Syntax:

<class_name> <object_name> = new <class_name>();

Basic Program – demo.java

```
class A1
{
        void fun1()
        {
                System.out.println("from A1");
        }
}
class A2
{
        void fun1()
        {
                System.out.println("from A2");
        }
}
class demo
{
        public static void main(String args[])
        {
        System.out.println("from main");
                A1 o1 = new A1();
                o1.fun1();
                A2 o2 = new A2();
                o2.fun1();
        }
}
```




```
G:\javaprog>javac demo.java

G:\javaprog>java demo
from main
from A1
from A2
```

Basic Program – with Fully Object Orientation

A1.java

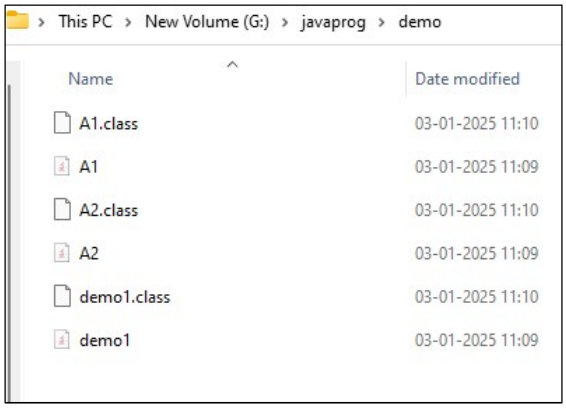
```
class A1
{
        void fun1()
        {
        System.out.println("from A1");
        }
}
```

A2.java

```
class A2
{
        void fun1()
        {
        System.out.println("from A2");
        }
}
```

demo1.java

```
class demo1
{
        public static void main(String
args[])
        {
        System.out.println("from main");
        A1 o1 = new A1();
        o1.fun1();
        A2 o2 = new A2();
        o2.fun1();
        }
}
```




```
G:\javaprog>cd demo

G:\javaprog\demo>javac demo1.java

G:\javaprog\demo>java demo1
from main
from A1
from A2

G:\javaprog\demo>
```

Data Types – primitive types

- **Integers** This group includes **byte**, **short**, **int**, and **long**, which are for whole-valued signed numbers.
- **Floating-point numbers** This group includes **float** and **double**, which represent numbers with fractional precision.
- **Characters** This group includes **char**, which represents symbols in a character set, like letters and numbers.
- **Boolean** This group includes **boolean**, which is a special type for representing true/false values.

Data Types – primitive types – Integers & Floating Point

Name	Width	Range
long	64	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	–2,147,483,648 to 2,147,483,647
short	16	–32,768 to 32,767
byte	8	–128 to 127

Name	Width in Bits	Approximate Range
double	64	4.9e–324 to 1.8e+308
float	32	1.4e–045 to 3.4e+038

VARIABLES

Declaration of Variables

type identifier [= *value*][, *identifier* [= *value*] …] ;

```
int a, b, c;
// declares three ints, a, b, and c.

int d = 3, e, f = 5;
// declares three more ints, initializing // d and f.

byte z = 22;
// initializes z.

double pi = 3.14159;
// declares an approximation of pi.

char x = 'x';
// the variable x has the value 'x'.
```

Dynamic Initialization

```
// Demonstrate dynamic initialization.

class DynInit
{
        public static void main(String args[])
        {
                double a = 3.0, b = 4.0;
                        // c is dynamically initialized
                double c = Math.sqrt(a * a + b * b);
                System.out.println("Hypotenuse is " + c);
        }
}
```

Variable Scope

```
class Scope
{
        public static void main(String args[])
        {
                int x; // known to all code within main
                x = 10;

                if(x == 10)
                { // start new scope
                        int y = 20; // known only to this block
                                        // x and y both known here.
                        System.out.println("x and y: " + x + " " + y);
                x = y * 2;
                }
        y = 100; // Error! y not known here
        // x is still known here.
        System.out.println("x is " + x);
        }
}
```

Type Conversion and Casting

```
// Demonstrate casts.
class Conversion {
public static void main(String args[]) {
byte b;
int i = 257;
double d = 323.142;

System.out.println("\nConversion of int to byte.");
b = (byte) i;
System.out.println("i and b " + i + " " + b);

System.out.println("\nConversion of double to int.");
i = (int) d;
System.out.println("d and i " + d + " " + i);

System.out.println("\nConversion of double to byte.");
b = (byte) d;
System.out.println("d and b " + d + " " + b);

}
}
```

OUTPUT

```
Conversion of int to byte.
i and b 257 1

Conversion of double to int.
d and i 323.142 323

Conversion of double to byte.
d and b 323.142 67
```

Type Promotion Rules in Expression

```
class Promote
{
public static void main(String args[])
{
byte b = 42;
char c = 'a';
short s = 1024;
int i = 50000;
float f = 5.67f;
double d = .1234;
double result = (f * b) + (i / c) - (d * s);
System.out.println((f * b) + " + " + (i / c) + " - " + (d * s));
System.out.println("result = " + result);
}
}
```

```
G:\javaprog>java Promote
238.14 + 515 - 126.3616
result = 626.7784146484375
```

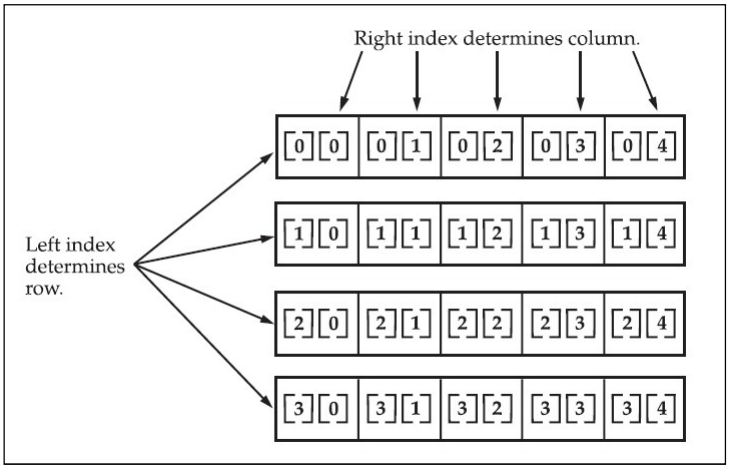
Arrays in JAVA

Arrays in JAVA

```
int month_days[];
month_days = new int[12];
month_days[0] = 31;
month_days[1] = 28;
month_days[2] = 31;
month_days[3] = 30;
month_days[4] = 31;
month_days[5] = 30;
month_days[6] = 31;
```

int month_days[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };

int twoD[][] = new int[4][5];



Escape Sequences

Escape Sequence	Description
\ddd	Octal character (ddd)
\uxxxx	Hexadecimal Unicode character (xxxx)
\'	Single quote
\"	Double quote
\\	Backslash
\r	Carriage return
\n	New line (also known as line feed)
\f	Form feed
\t	Tab
\b	Backspace